# CSCI 112 - Lab: Understanding Makefiles

**Duration:** 1.5 hours (90 minutes)

**Learning Objectives:** - Understand Makefile syntax and structure - Build projects with multiple source files - Use dependencies and automatic compilation - Organize projects with directories - Master compiler and linker flags

---

## Task 1: The Simplest Makefile (10 min)

### Scenario

You have a single C file and need to compile it.

### Directory Structure

```
Task1/
  main.c
  Makefile
```

### Files to Create

**main.c:**

```c
#include <stdio.h>

int main()
{
    printf("Hello from Task 1!\n");
    return 0;
}
```

### Your Task

1. Create a `Makefile` with these targets:
   - `compile` - compiles main.c to executable `program`
   - `run` - executes the program
   - `clean` - removes the executable
2. Test each target:

```
make compile
make run
make clean
```

### Expected Output

```
make compile: Creates 'program' executable
make run: Prints "Hello from Task 1!"
```

`make clean: Removes 'program'`

### Solution

```
CC = gcc
CFLAGS = -Wall

compile:
    $$(CC) $$(CFLAGS) main.c -o program

run: compile
    ./program

clean:
    rm -f program

.PHONY: compile run clean
```

---

## Task 2: Multiple Source Files (15 min)

### Scenario

Project has multiple `.c` files. Compile them together without dependencies.

### Directory Structure

```
Task2/
  main.c
  math.c
  math.h
  Makefile
```

### Files to Create

**main.c:**

```c
#include <stdio.h>
#include "math.h"

int main()
{
    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);
    return 0;
}
```

**math.h:**

```c
#ifndef MATH_H
#define MATH_H

int add(int a, int b);

#endif
```

**math.c:**

```c
int add(int a, int b)
{
    return a + b;
}
```

### Your Task

1. Create a `Makefile` that:
   - Compiles `main.c` and `math.c` together
   - Creates executable `calculator`
   - Has targets: `build`, `run`, `clean`
2. Test:

```
make build
make run
make clean
```

### Expected Output

```
make build: Creates 'calculator' executable
make run: Prints "5 + 3 = 8"
make clean: Removes 'calculator'
```

### Solution

```
CC = gcc
CFLAGS = -Wall -std=c99

build:
    $$(CC) $$(CFLAGS) main.c math.c -o calculator

run: build
    ./calculator

clean:
    rm -f calculator

.PHONY: build run clean
```

---

## Task 3: Object Files & Dependencies (20 min)

### Scenario

Compile to object files first, then link. Recompile only changed files.

### Directory Structure

```
Task3/
  main.c
  math.c
  math.h
  Makefile
```

### Files to Create

**main.c, math.c, math.h:** (same as Task 2)

### Your Task

1. Create a `Makefile` that:
   - Compiles `main.c` → `main.o`
   - Compiles `math.c` → `math.o`
   - Links `main.o` + `math.o` → `calculator`
   - Only recompiles `.c` files if they change
   - Has targets: `build`, `run`, `clean`
2. Test changes:

```
make build
touch math.c
make build    # Should only recompile math.c and relink
```

### Expected Output

```
First make build: Compiles both files, links executable
Second make build: Only recompiles math.o and relinks
```

### Hint

Use pattern rules:

```
%.o: %.c
    $$(CC) $$(CFLAGS) -c $$< -o $$@
```

### Solution

---

## Task 4: Pattern Rules & Auto Dependencies (20 min)

### Scenario

Simplify Makefile using pattern rules. Use `-MMD` for automatic dependency generation.

### Directory Structure

```
Task4/
  main.c
  math.c
  math.h
  Makefile
```

### Files to Create

(same as Task 3)

### Your Task

1. Create a `Makefile` that:
   - Uses pattern rule `%.o: %.c` for all `.o` files
   - Uses `-MMD` flag to auto-generate `.d` dependency files
   - Includes `.d` files so recompilation happens automatically
   - Has targets: `build`, `run`, `clean`
2. Verify auto-dependency:

```
make build
cat main.d      # Shows dependencies
```

### Expected Output

```
Generated main.d contains: main.o: main.c math.h
If math.h changes, main.o is automatically recompiled
```

### Solution

---

## Task 5: Directory Organization with Automatic Creation (25 min)

### Scenario

Organize project: `src/` for source, `bin/` for executables, `obj/` for object files. Makefile should create missing directories.

### Directory Structure

```
Task5/
  Makefile
  src/
      main.c
      math.c
      math.h
```

### Files to Create

**src/main.c, src/math.c, src/math.h:** (same as previous tasks)

### Your Task

1. Create a `Makefile` that:
   - Compiles `src/*.c` to `obj/*.o`
   - Links to `bin/calculator`
   - **Automatically creates `bin/` and `obj/` directories**
   - Uses pattern rules
   - Has targets: `build`, `run`, `clean`, `clean-all`
2. Test directory creation:

```
make build      # Should create bin/ and obj/ automatically
ls -la bin/ obj/
```

### Expected Output

```
bin/ and obj/ directories created automatically
bin/calculator executable created
obj/main.o and obj/math.o created
```

### Hint

Use `|` (order-only prerequisites) to create directories:

```
obj/%.o: src/%.c | obj
    $$(CC) $$(CFLAGS) -c $$< -o $$@

obj:
    mkdir -p obj
```

### Solution

---

## Task 6: Compiler & Linker Flags with gcc -std=c99 (25 min)

### Scenario

Use proper compiler flags, linker flags, and multiple configurations.

### Directory Structure

```
Task6/
  Makefile
  src/
      main.c
      math.c
      math.h
```

### Files to Create

**src/main.c:**

```c
#include <stdio.h>
#include <math.h>
#include "math.h"

int main()
{
    int sum = add(5, 3);
    double sqrt_val = sqrt(16.0);

    printf("5 + 3 = %d\n", sum);
    printf("sqrt(16) = %f\n", sqrt_val);

    return 0;
}
```

**src/math.c, src/math.h:** (same as previous tasks)

### Your Task

1. Create a `Makefile` with:
   - `CC = gcc`
   - `CFLAGS = -Wall -Wextra -std=c99 -O2`
   - `LDFLAGS = -lm` (link math library)
   - Directories: `src/`, `obj/`, `bin/`
   - Auto-dependency generation with `-MMD`
   - Targets: `build`, `debug`, `run`, `clean`, `clean-all`
2. Two configurations:
   - `make build` - optimized with `-O2`

- `make debug` - with `-g` for debugging

3. Test:

```
make debug
make build
make run
```

### Expected Output

```
make debug: Compiled with debugging symbols
make build: Compiled with optimization
make run: Executes and shows correct output
```

### Solution

---

## Bonus: Task 7 (if time permits - 15 min)

### Challenge: Wildcard Rules & Recursive Compilation

Create a `Makefile` that:

```
SOURCES = $$(wildcard $$(SRCDIR)/*.c)
OBJECTS = $$(SOURCES:$$(SRCDIR)/%.c=$(OBJDIR)/%.o)
```

This automatically compiles ALL `.c` files in `src/` directory without listing them!

### Modified Files

Add new file **src/utils.c:**

```
int multiply(int a, int b)
{
    return a * b;
}
```

Update **src/main.c** to use it and add `utils.h` declaration.

### Solution

---

## Summary: Makefile Progression

| Task | Concept | Time |
|---|---|---|
| 1 | Basic compilation | 10 min |
| 2 | Multiple files | 15 min |
| 3 | Object files & dependencies | 20 min |
| 4 | Pattern rules & auto-deps | 20 min |
| 5 | Directory organization | 25 min |
| 6 | Compiler flags & configurations | 25 min |
| 7 | Wildcard rules (bonus) | 15 min |

**Total: 90+ minutes**

---

## Key Takeaways

**Task 1:** Variables and basic targets
**Task 2:** Multiple source files
**Task 3:** Object files reduce compilation time
**Task 4:** Auto-dependencies prevent bugs
**Task 5:** Directory organization with `mkdir -p` and `|`
**Task 6:** Compiler flags (`-Wall -std=c99 -O2`) and linker flags (`-lm`)
**Task 7:** Wildcards automate compilation